ELISA
Enabling Linux in Safety Applications

SEMINAR

23 July 2026
7AM PT / 2PM UTC / 4PM CET

WHAT-WHY-HOW: A practical model for understanding software requirements

Stanislav Pankevich
Lead Software Engineer
Reflex Aerospace GmbH

What is this about?

This presentation is a brief overview of software systems engineering and software requirements through the lens of the WHAT-WHY-HOW model.

It should be useful for people who are both familiar with and new to these topics.

See also a related ELISA webinar:

"Introduction to Requirements Engineering" by Pete Brink

What is WHAT-WHY-HOW?

- A simple but practical mental model
- Useful for thinking about technical artifacts and workflows
- Easy to recognize in daily work after learning the key concepts
- This webinar provides:
  - A description of the model in relation to familiar systems engineering artifacts
  - Heuristics for identifying the WHAT, WHY, and HOW aspects
  - A description of common pitfalls
- Benefits:
  - A practical framework for analyzing engineering artifacts
  - A shared vocabulary for technical discussions
  - Better understanding of requirements, architecture, and documentation
  - Earlier identification of some systems engineering anti-patterns
  - Improved communication between customers, systems engineers, and developers

Background for this work

- Satellite software development at Reflex Aerospace GmbH
  - Disclaimer: The views expressed are my own and not those of my employer
- StrictDoc, open source tooling for requirements and technical documentation
- Participation as guest in ELISA Safety Architecture and SPDX FuSa WGs
- Inspirations from ECSS, DO-178, DOD, ISO/IEC 15288, SWEBOK
- Practical heuristics, not formal knowledge. Feedback is appreciated!

Thanks to:

Greg Shue, Tobias Deiminger, Nicole Pappler, and Kate Stewart for feedback and support!

Agenda

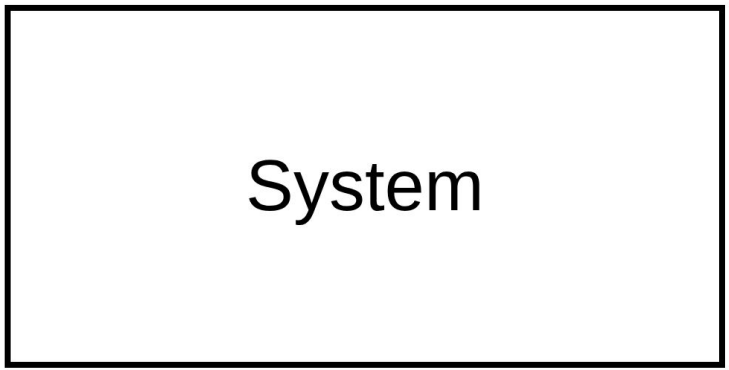
- Basic building blocks for systems engineering
- The WHAT-WHY-HOW model: Simple and recursive versions
- Common WHAT-WHY-HOW pitfalls
- Practical takeaways

Basic building blocks for systems engineering

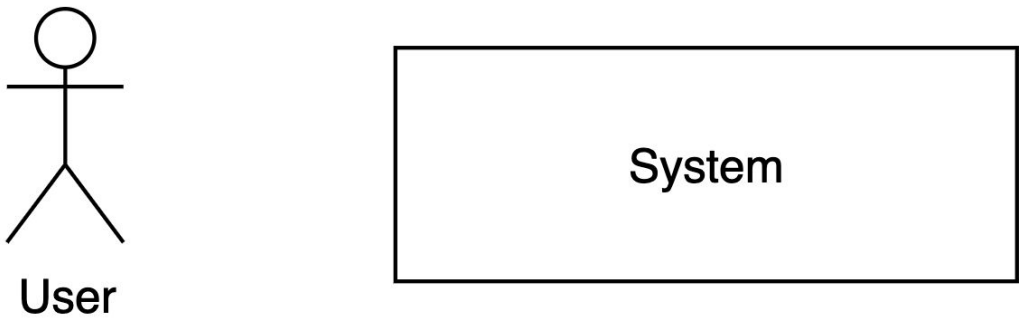
ELISA
Enabling Linux in Safety Applications

SEMINAR

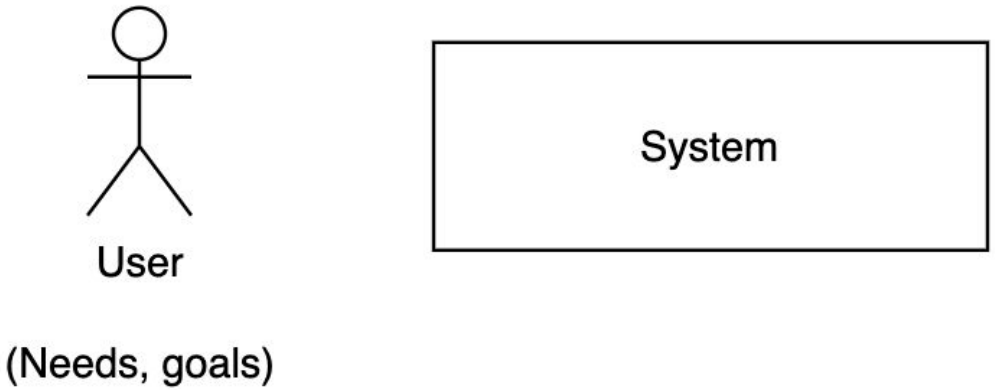
Let's talk about a system



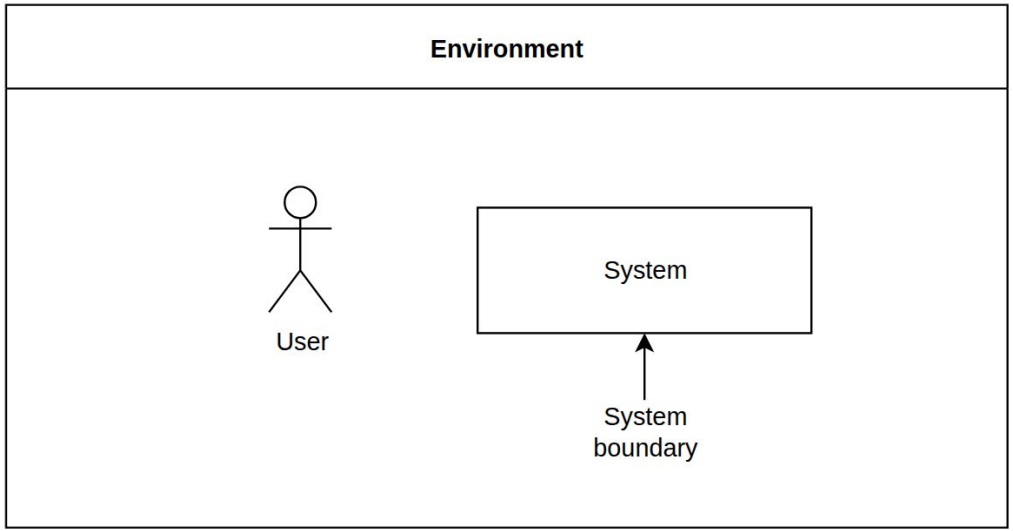
The system has a user



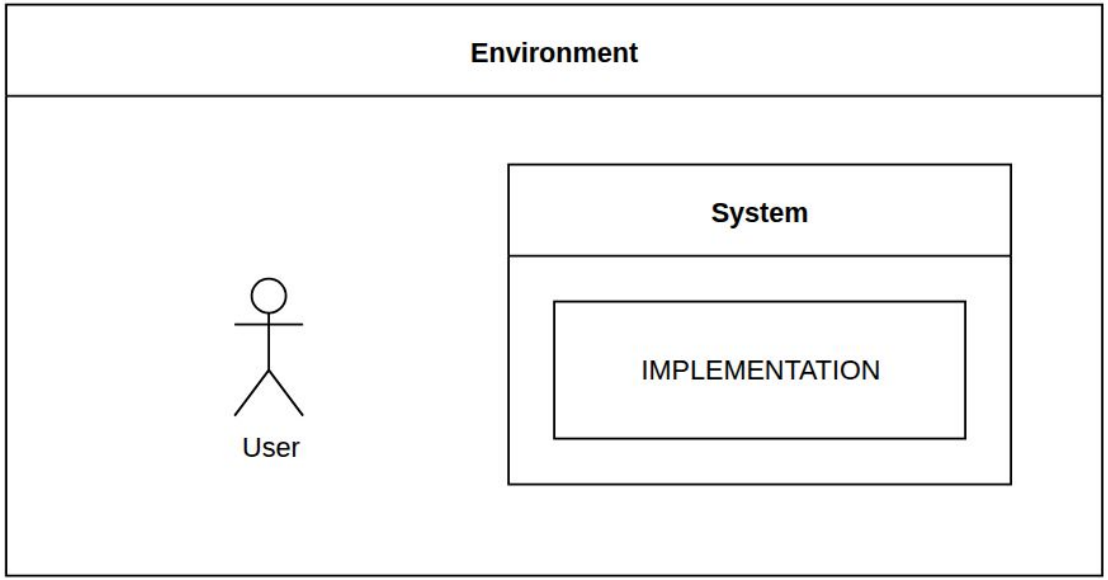
The user has needs/goals that the system can address



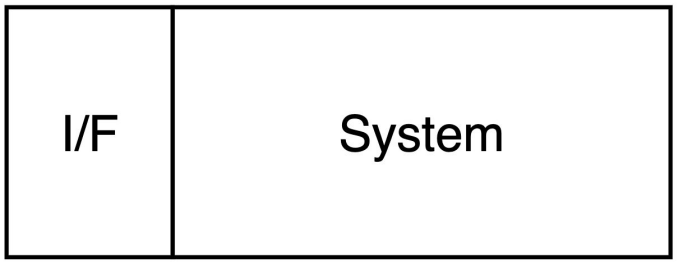
System environment



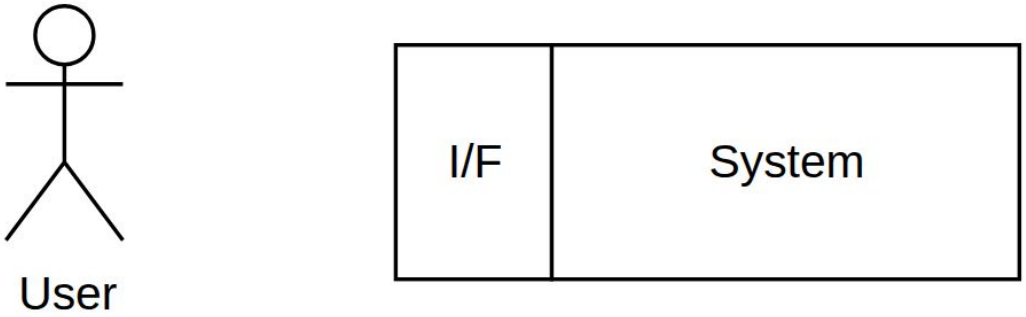
System implementation



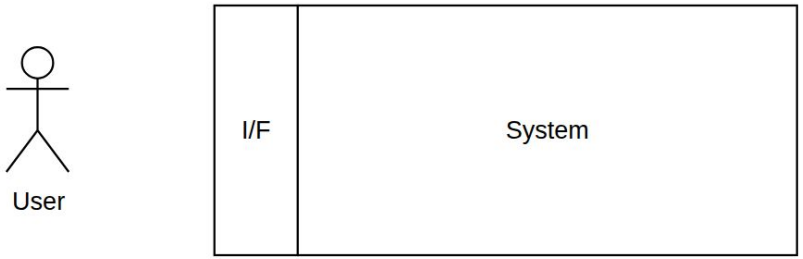
A system usually has an interface



A user uses the interface to control/access the system



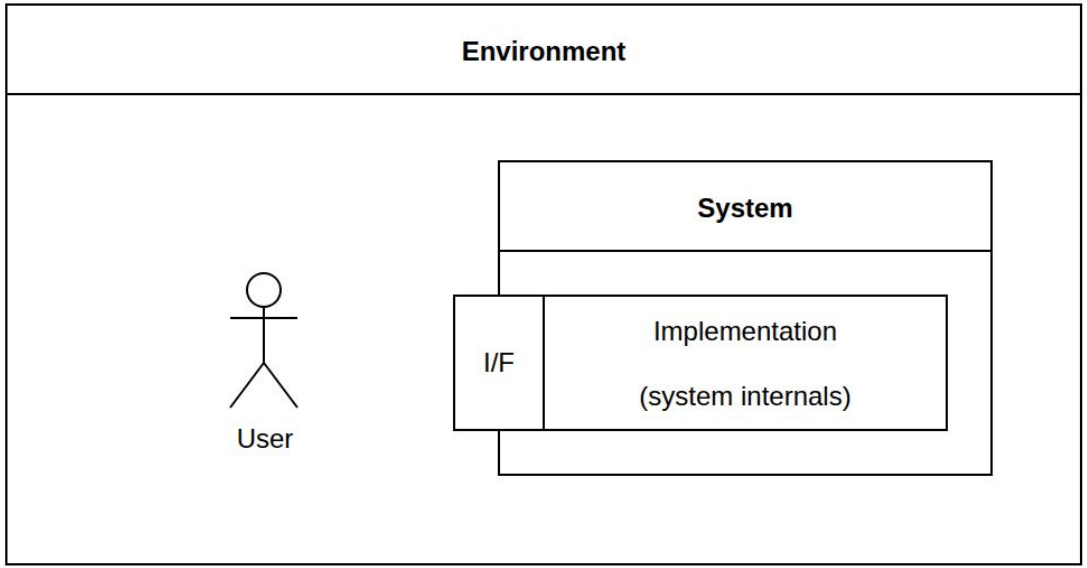
Interface hides/encapsulates complexity



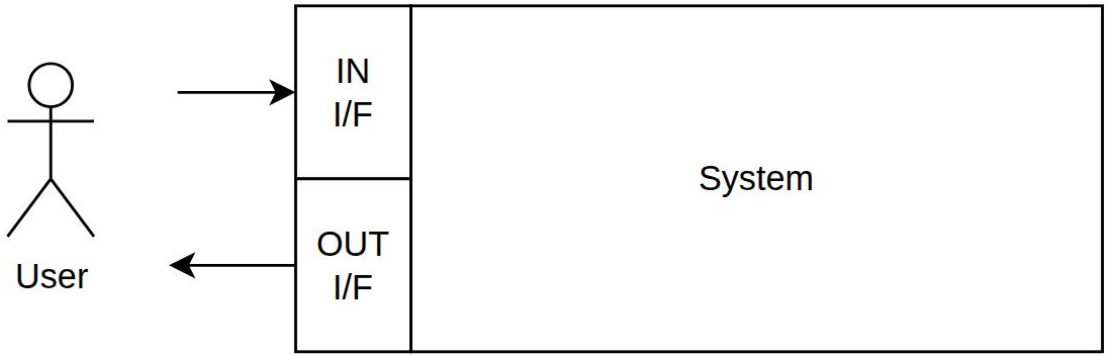
Small interface, rich functionality:

A good interface is small and simple, exposing only the minimal surface needed for the user to interact with the system. Behind it, the system implementation can be large and complex, encapsulating the internal logic while delivering substantial value through that small interface.

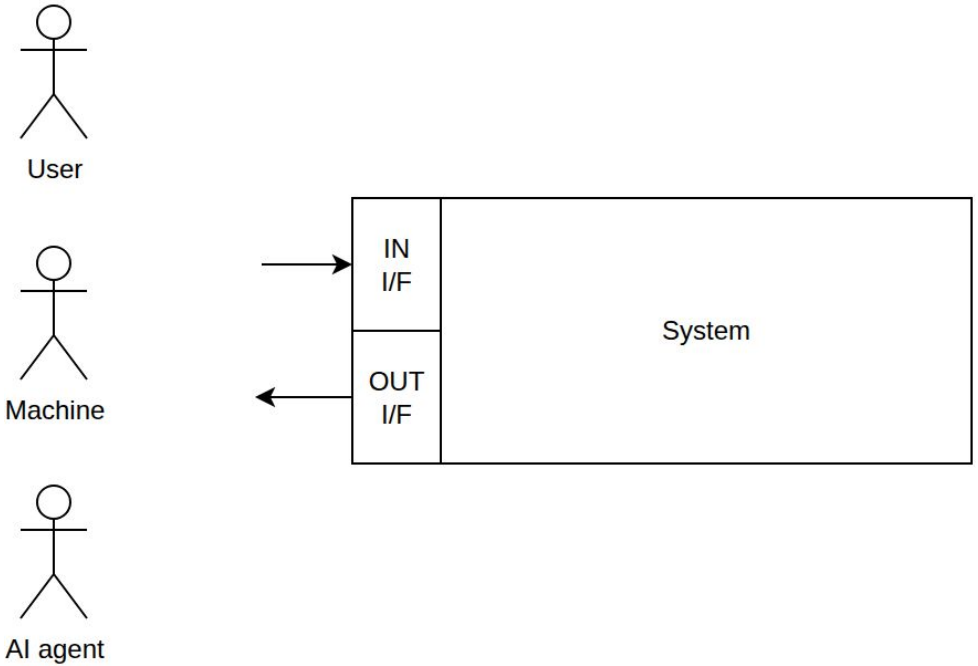
Interface is what connects a user with the system



Interface I/O model

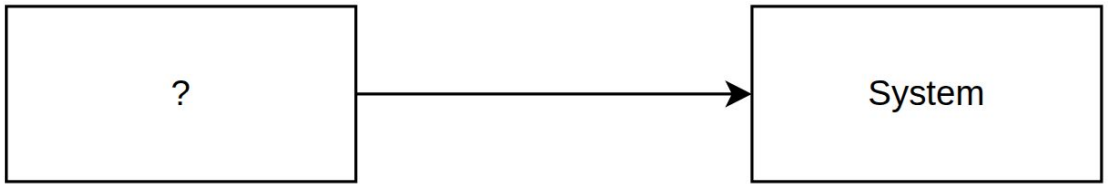


A user can be a human or another system

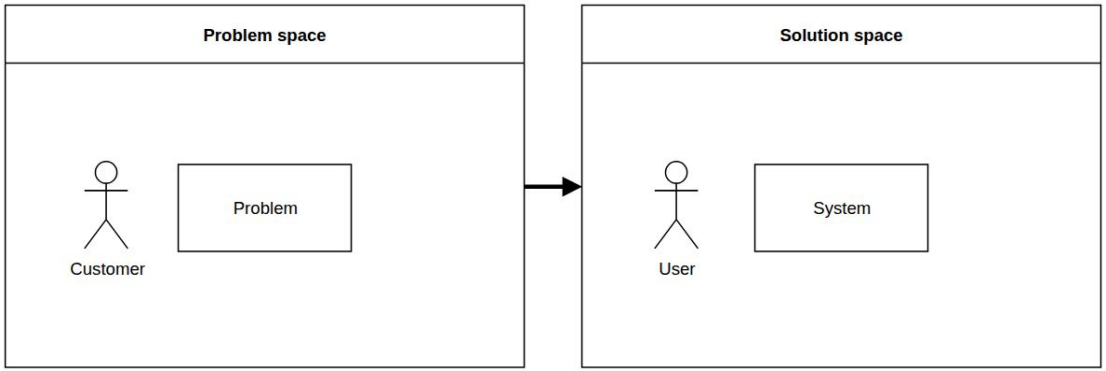


Problem space vs solution space

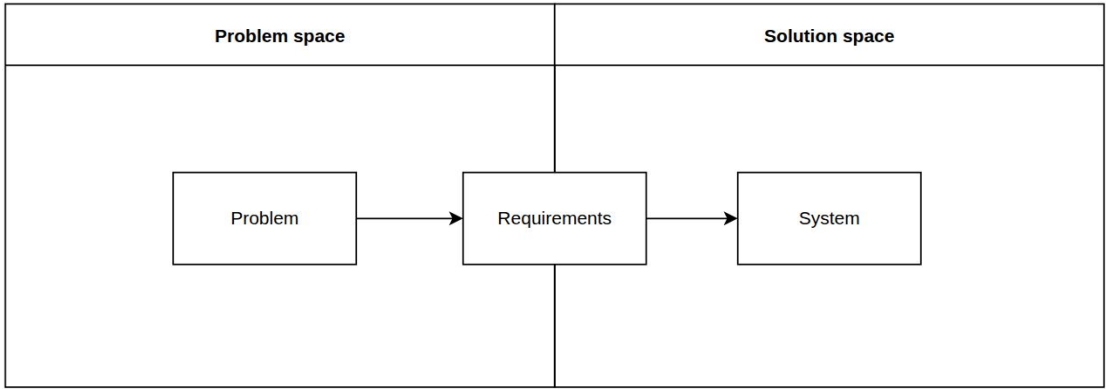
Where does the system come from?



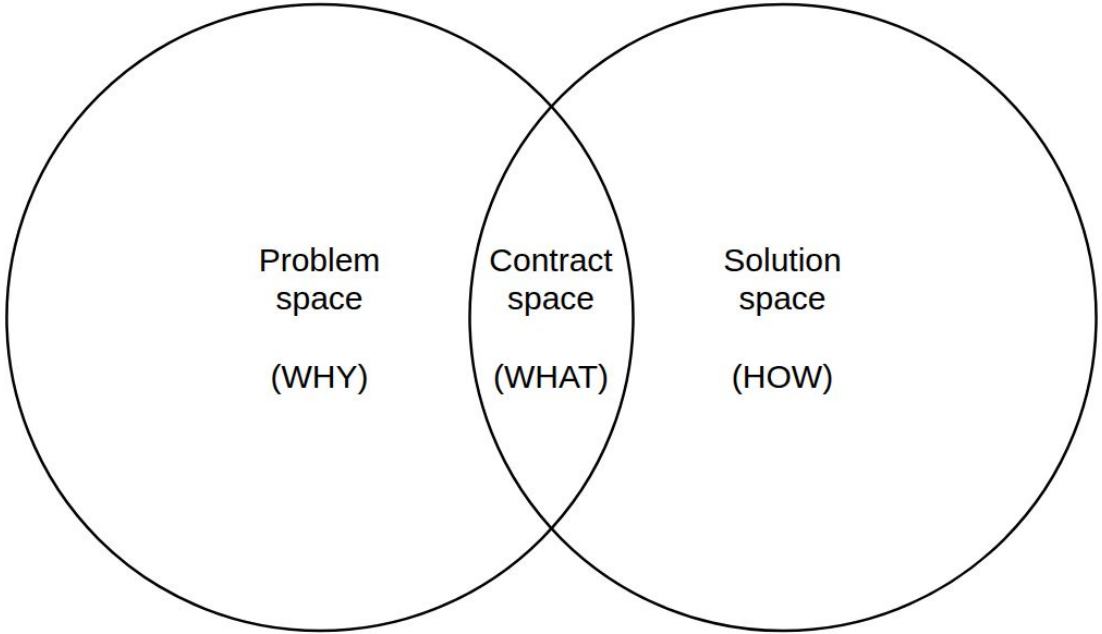
Problem space vs solution space



Problem → Requirements → Solution

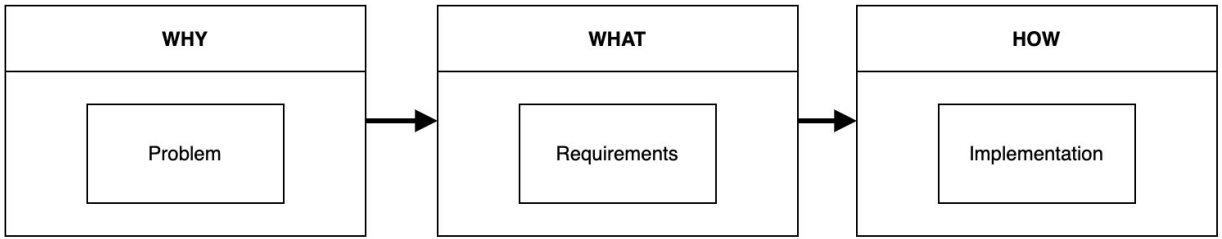


Problem space vs contract space vs solution space

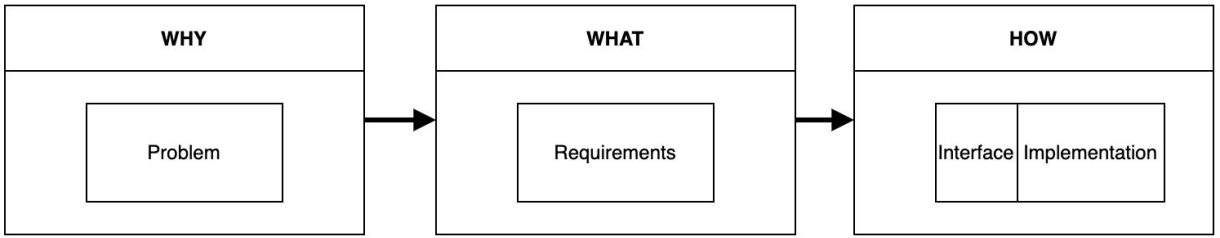


WHAT-WHY-HOW

Let's not get lost! It is really about WHAT-WHY-HOW!



WHY→WHAT→HOW (and INTERFACE)



Why WHAT-WHY-HOW, not WHY-WHAT-HOW?

Our attention usually starts with WHAT (the main focus point):

1) WHAT am I working on now?
2) WHY am I working on it?
3) HOW are we going to do it?

Seen from this perspective, the WHAT-WHY-HOW is an IO device itself:

WHAT is processing, WHY is input, HOW is output

WHAT-WHY-HOW — Easy? Or not really?

- People generally agree that all four aspects of WHAT-WHY-HOW-IF exist, but they are not always treated separately and are often mixed together.

- Separating these aspects may look simple, but in practice it is not easy. In engineering, WHAT and WHY often receive less attention than HOW. Open source tools also provide less support for WHAT and WHY.

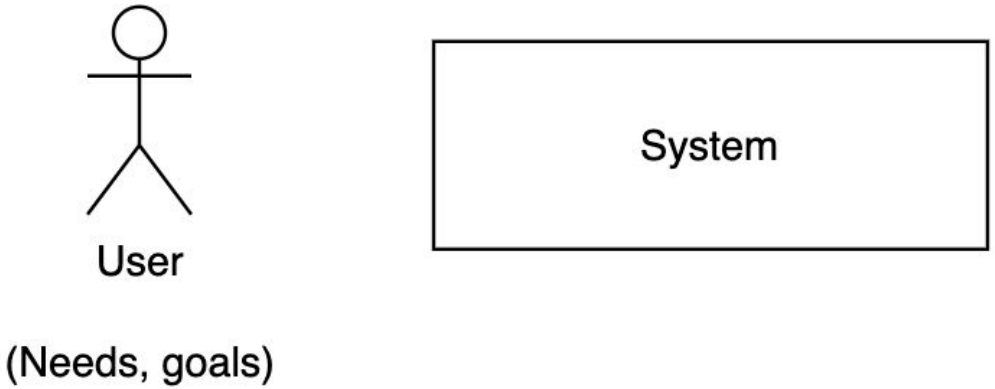
Let's examine these aspects more closely and see if we can separate them.

WHY

WHY: Solving Right problems

> *"Engineers are great at solving problems*
> *but they are not always great at identifying the right problems to be solved."*
> *Dr. John Thomas, ESWC 2019*

The user has needs (WHY) that the system can address

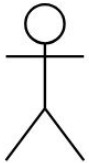


Problem (WHY) exists independently of any solution

Problem space:

- Problem of Environment
- User needs
- Motivation
- Problem statement
- Business goals
- Describes a need, undesired state, "Pain", pressure from reality.

WHY — Opportunities for solutions

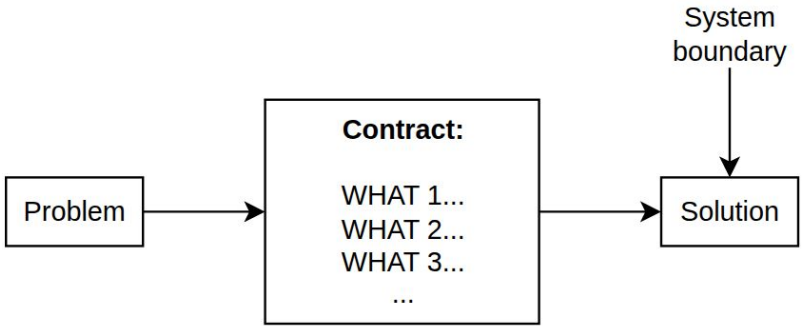


Problem space:

- Describes problem without hinting at solution
- Oppotunities for solutions
- Can have multiple solutions, has no obligations
- Only intention to create solution, without requirements
- Describes a problem/situation without system
- No system, no system boundary

WHAT

WHAT — Contract for system's behavior a.k.a Requirements



Contract space:

- Formal requirements
- "Shall" statements
- Measurable obligations
- Criteria that the solution must meet
- Introduces system boundary
- Reduces solution space
- Multiple solutions possible

Requirement statement/rationale examples

STATEMENT example — WHAT

*The sem_wait() function **shall lock the semaphore** referenced by sem by performing a semaphore lock operation on that semaphore. If the semaphore value is currently zero, then…*

https://pubs.opengroup.org/onlinepubs/9799919799/functions/sem_wait.html

Ideally, requirements should also have a **`UUID`** for easier linking to other elements.

RATIONALE example — WHY

*The nanosleep() function specifies that the system-wide clock CLOCK_REALTIME is used to measure the elapsed time for this time service. However, with the introduction of the monotonic clock CLOCK_MONOTONIC **a new relative sleep function is needed** to allow an application to take advantage of the special characteristics of this clock.*

https://pubs.opengroup.org/onlinepubs/9799919799/functions/clock_nanosleep.html

Example: WHAT-WHY in requirement

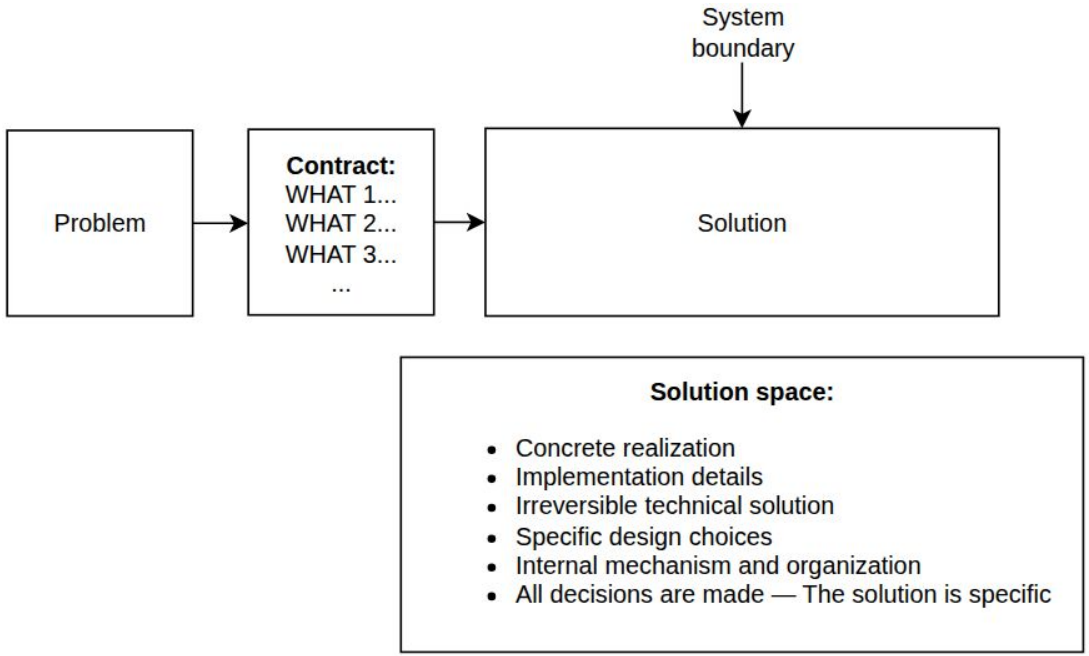
WHAT

WHY

Title: Avionics thermal control	
UID:	REQ-345
Statement:	The spacecraft avionics module shall maintain an operating temperature between −10 °C and +50 °C during all nominal mission phases.
Parents:	REQ-123
Rationale:	The parent requirement specifies that the electronic components in the avionics module shall operate within this temperature range. Exceeding these limits may result in degraded performance or permanent component failure.

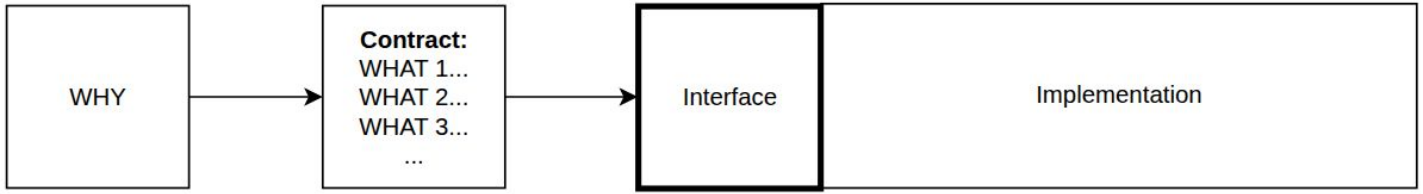
HOW

HOW — Implementation details



INTERFACE

HOW-Interface vs HOW-Implementation

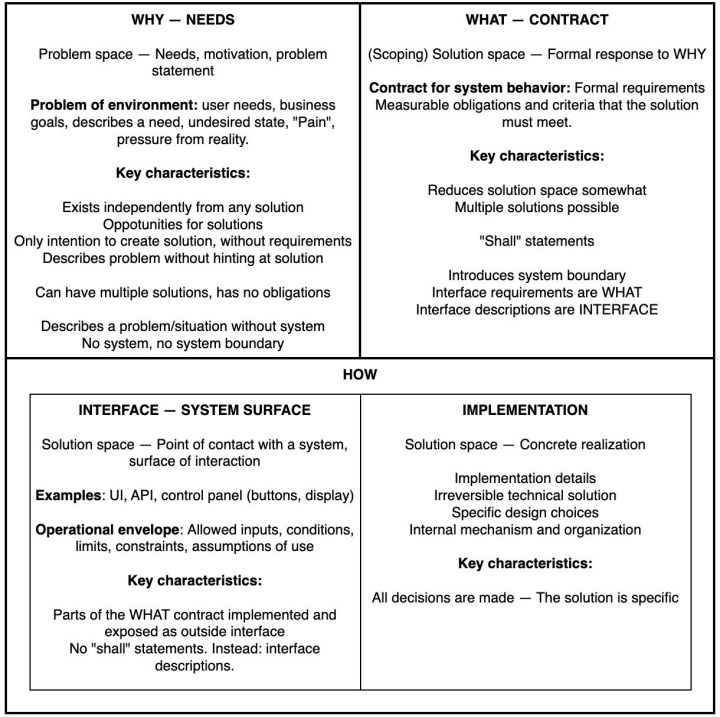


Solution space (Interface):

- Point of contact with a system, surface of interaction
- Operational envelope: Allowed inputs, conditions, limits, constraints, assumptions of use
- Parts of the WHAT contract implemented and exposed as outside interface
- No "shall" statements. Instead: interface descriptions
- Examples: GUI, API, Control panel (buttons, display)

WHAT-WHY-HOW summary so far

WWH(I) summary



WWH(I) consistency principle a.k.a. WWH Onion

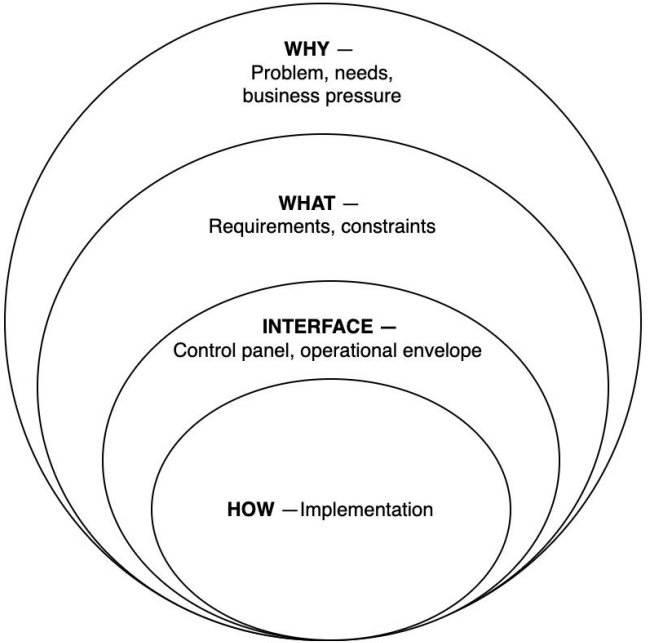
WWH consistency principle: A system development is consistent if all layers are represented and the direction of influence is preserved:

WHY defines the intent that drives all other layers.

WHAT operationalizes that intent into the outcome to be achieved through HOW while remaining aligned with WHY.

INTERFACE exposes the system in a way that reflects HOW while hiding unnecessary implementation details.

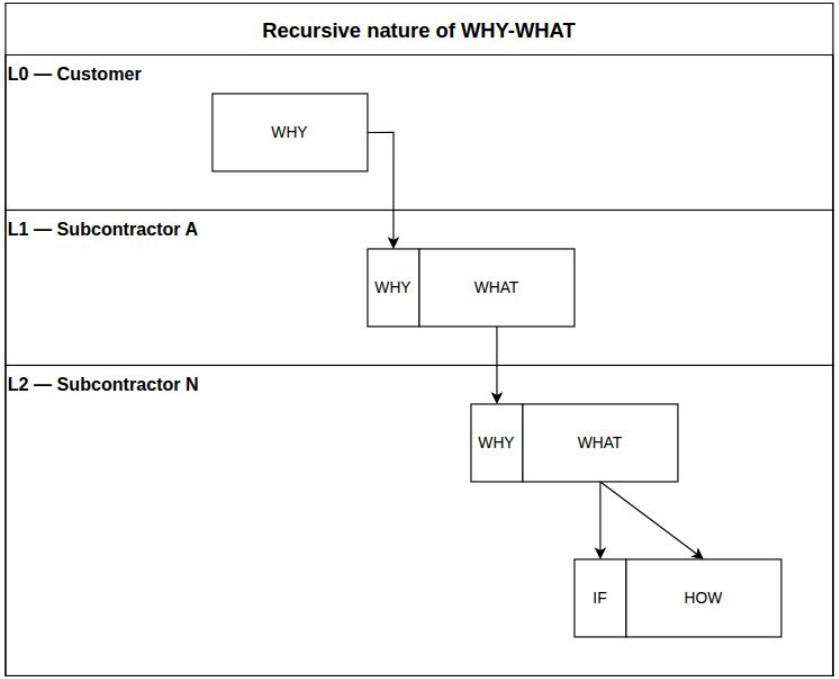
HOW implements the system strictly according to WHAT and WHY, and nothing else.



Recursive WHY-WHAT-HOW

Recursive WHAT-WHY-HOW

- Every level delegates its HOW to next levels
- Each new level is a HOW for the previous level
- The last levels are the actual HOW-implementor



Abstraction level breakdown example

Missing delegation of HOW to the next level:

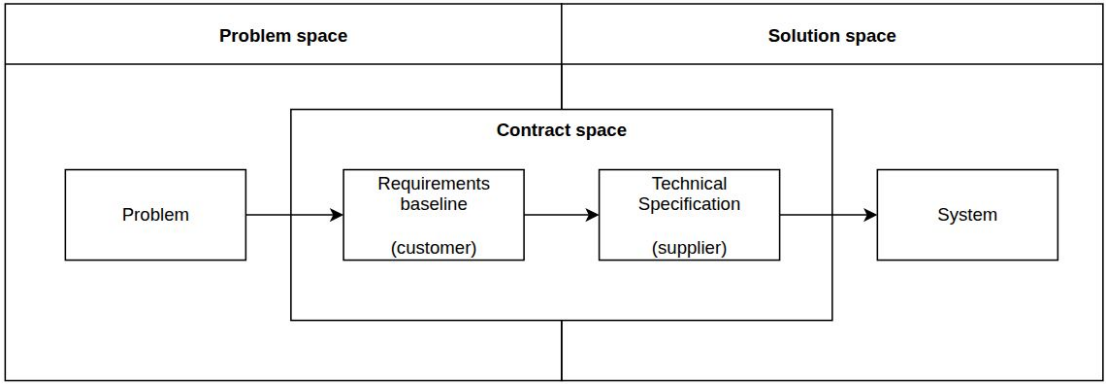
→ WHAT absorbs implementation details

→ Too much "HOW"-details

→ Loss of clarity and traceability to WHY

Maintaining a large project's requirements in one level of specification is very difficult. Some delegation is needed to provide for clarity and maintainability.

Contract space — European space standard for SW (ECSS-E-ST-40)



Customer baseline → Supplier specification (ECSS)

Unclear responsibilities assignment between system and software teams may result into software failures. In particular the interpretation of the “system needs” by the software teams is the cornerstone of the system and software processes.

A key principle of the ECSS-E-ST-40C to achieve as far as possible the common understanding of the software requirements, is to **build the requirements in two steps (RB for needs then TS for solution) and to perform a cross-check through two separate requirements documents**, i.e. Requirements Baseline (RB) and Technical Specification (TS), and reviews:

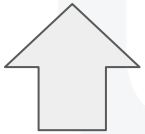
- System Requirements Review to check the Needs completeness

  and

- the Preliminary Design Review (potentially anticipated by the Software Requirements Review) to allow:
  - the Customer checking that the proposed software solution answers the needs
  - the software supplier checking that his understanding of the customer needs are correct, complete and results into a feasible software solution.

This handbook also recommends **an early integration of the software engineering process in the system engineering phase (co engineering)** to facilitate the common understanding of the requirements: the supplier can early understand the software needs and checks their feasibility, before the SRR.

ECSS-E-HB-40A, 11 December 2013



Software systems engineering

Requirement flowdown: (WHAT-WHY) → (WHAT-WHY) → …

Mission requirements

↓ (system definition)

System requirements (satellite)

↓ (subsystem decomposition)

Subsystem requirements (Satellite subsystems, e.g., Data handling, Power, Thermal, AOCS, Payload)

↓ (software functional decomposition)

High-level software requirements (logical SW functions, e.g., Mission management, File management, etc.)

↓ (software architecture and design)

Low-level software requirements (physical SW components, e.g., Mission Manager App, File Manager App)

High-level vs low-level requirements

DO-178B [27] specifies that the software requirements should be organized into high-level and low-level software requirements. Also, the high-level software requirements should comply with the system requirements (objective 1 of table A-3 in appendix A), and the low-level software requirements should comply with the high-level software requirements (objective 1 of table A-4 in appendix A). These are defined in DO-178B as:

- *"High-level requirements: Software requirements developed from analysis of system requirements, safety-related requirements, and system architecture.*
- *Low-level requirements: Software requirements derived from high-level requirements, derived requirements, and design constraints from which source code can be directly implemented without further information."*

DOT/FAA/AR-08/32 Requirements Engineering Management Handbook

In general, HLR represent "what" is to be designed and LLR represent "how" to carryout the design. Merging these two levels leads to several certification concerns, as described in this paper.

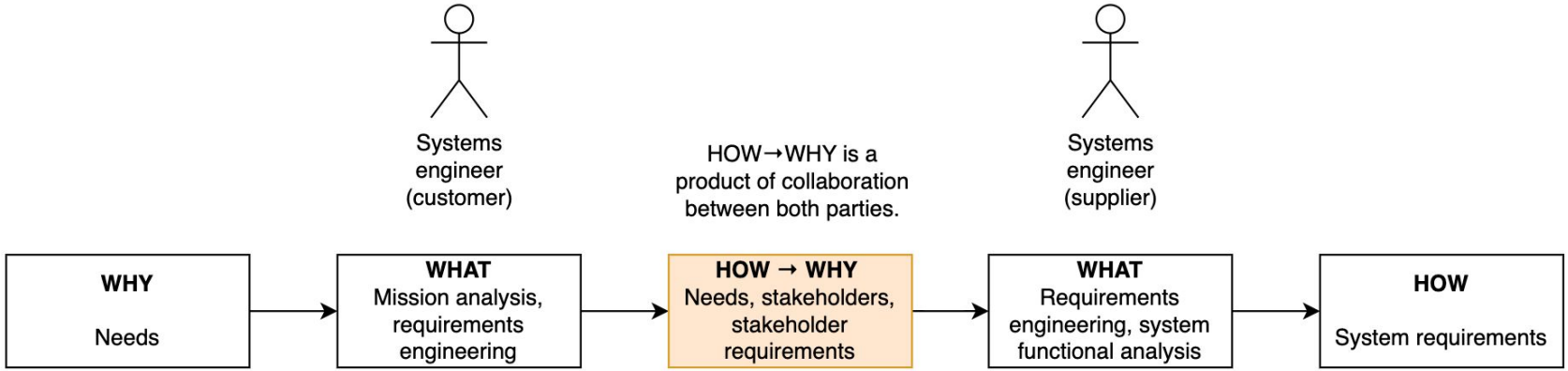
Position Paper CAST-15 Merging High-Level and Low-Level Requirements

A heuristic for what low-level software requirements are

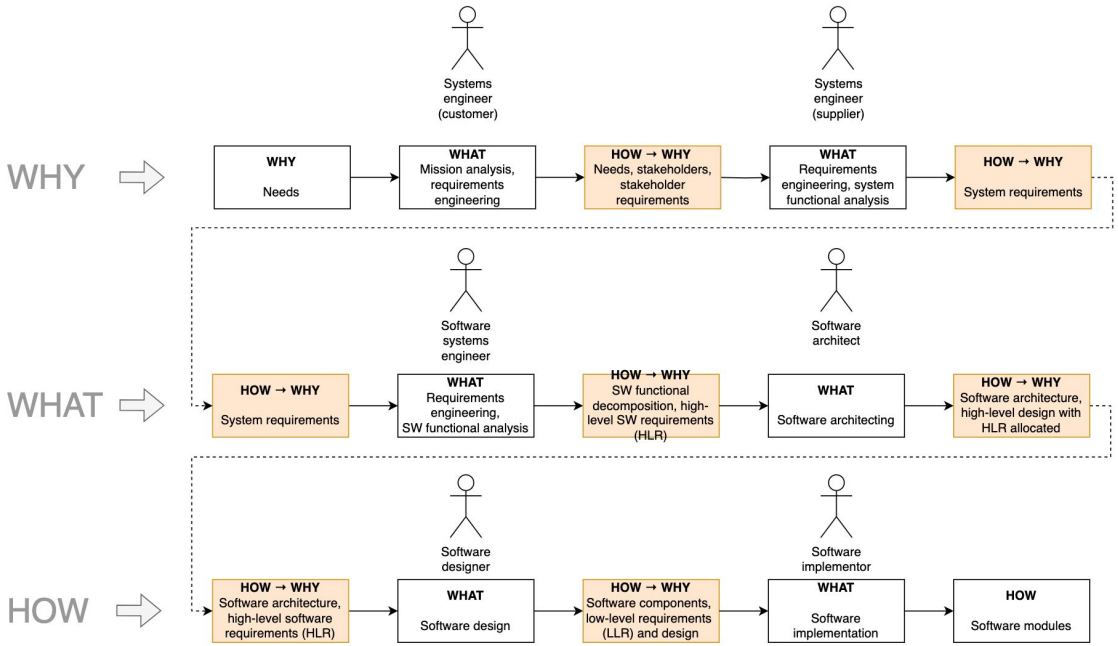
"...Low-level requirements: Software requirements derived from high-level requirements, derived requirements, and design constraints from which source code can be directly implemented without further information." (DO-178C)

An easy heuristic: low-level requirements are everything that AI would need to implement a solution without asking any further information.

Recursive WHAT-WHY-HOW in Systems Engineering



Recursive WHAT-WHY-HOW in Software Engineering



Think of WHAT-WHY-HOW as a transparent overlay with three cut-outs. You can place it over any engineering discipline, artifact, or process. The labels remain the same, but what appears through each opening changes depending on what the overlay is applied to.

WHAT-WHY-HOW in documentation

WWH(I) applied to industry documents (examples)

WHY	WHAT
Business Case Product Brief Product Vision Stakeholder needs Statement of Work Concept of Operations (ConOps) (Top-level) Mission requirements	Project Requirements Functional Specification Technical Requirements Specification Software Requirements Specification High-Level Requirements (HLR) Low-Level Requirements (LLR) Interface Requirements Document (IRD)
INTERFACE	**HOW**
Interface Control Document (ICD) API reference documentation Operator Manual User Manual Safety Manual Instructions manual Flight Operations Manual	Design Definition File (DDF) Design Justification File (DJF) Software Design Description (SDD) Software Design Document (SDD) Architecture Design Document (ADD) Architecture Decision Records (ADR)

NOTE: Understanding the recursive nature of WHAT-WHY-HOW is important.

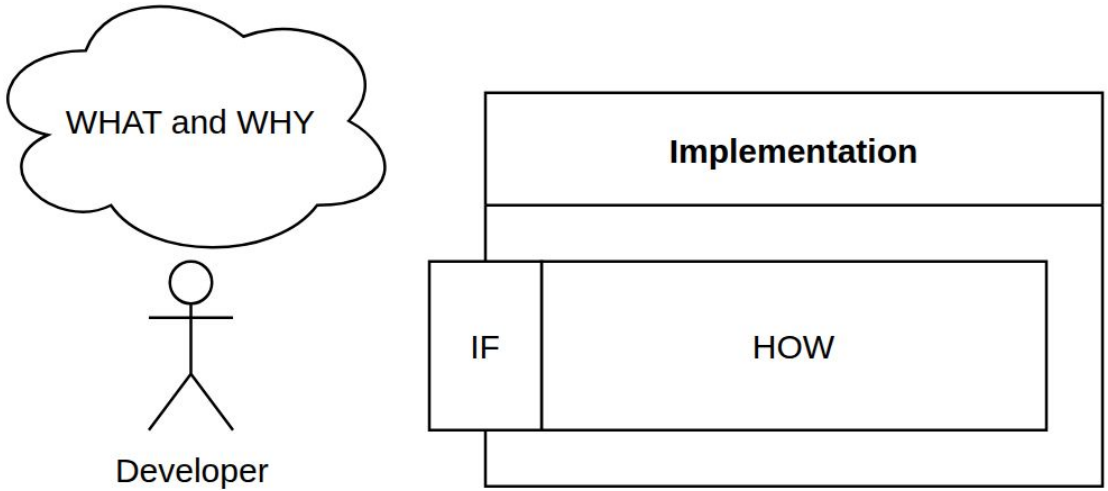
Depending on the project level breakdown and customer-supplier systems co-engineering, some WHY documents can also act like WHAT and vice versa.

Example: The lower-level CONOPS documents can also exist in WHAT.

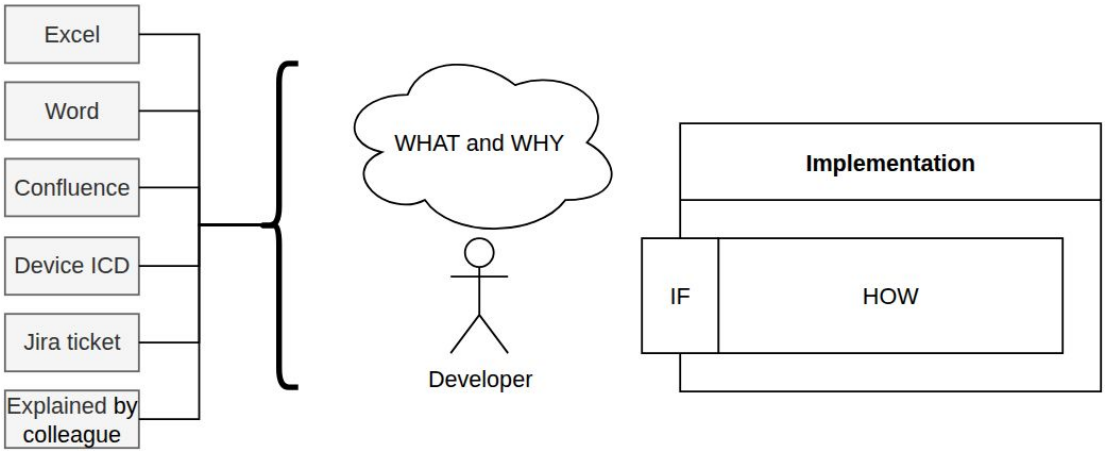
Software development with WHAT-WHY-HOW

SEMINAR

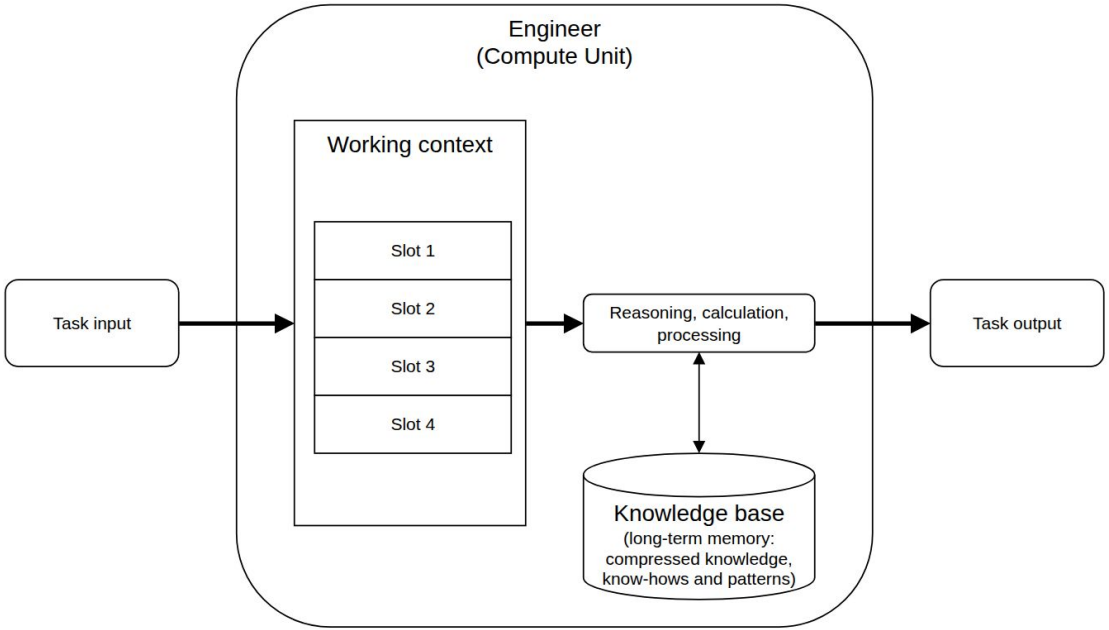
Software development with WHAT-WHY-HOW



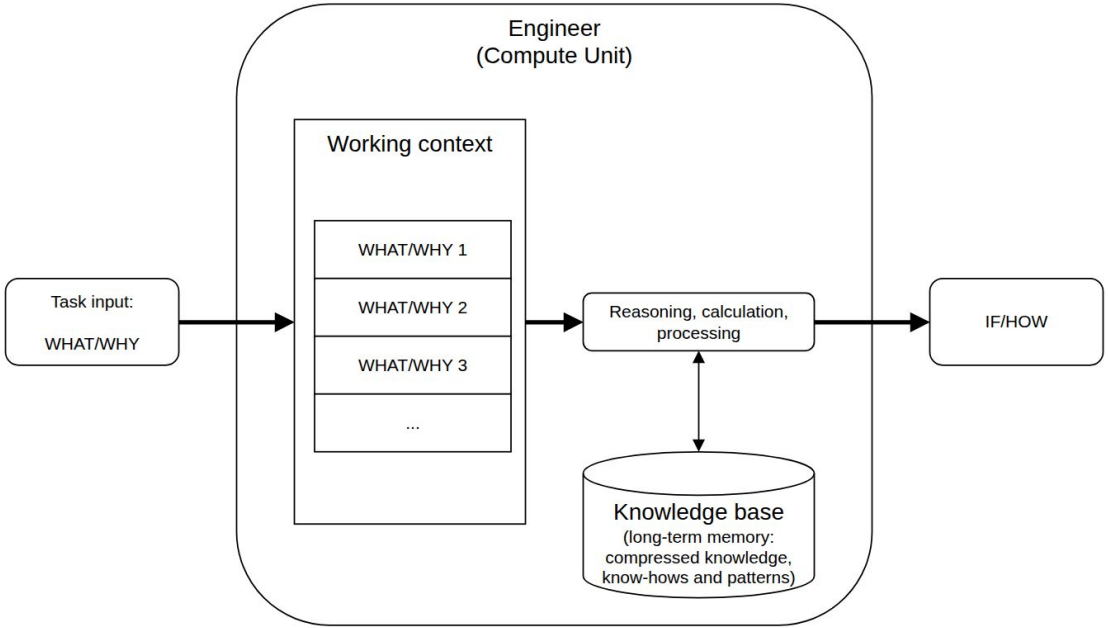
Software development with WHAT-WHY-HOW — Inputs



Engineer as a Compute Unit



Engineer as a WHAT-WHY-HOW compiler



WHAT-WHY-HOW: Common pitfalls

Pitfall 1: No WHY

Classics! WHY is lost somewhere in the documentation or project history.

When the WHY is lost, we are left with instructions without intent. We see them, but we cannot easily verify whether they are still the right thing to develop, maintain, adapt them to new situations, or challenge outdated assumptions.

Example: Inherited software: a team may maintain a workaround, interface constraint, or performance requirement for years, but if the original reason is gone, nobody knows whether removing it would break the system or simply remove obsolete complexity.

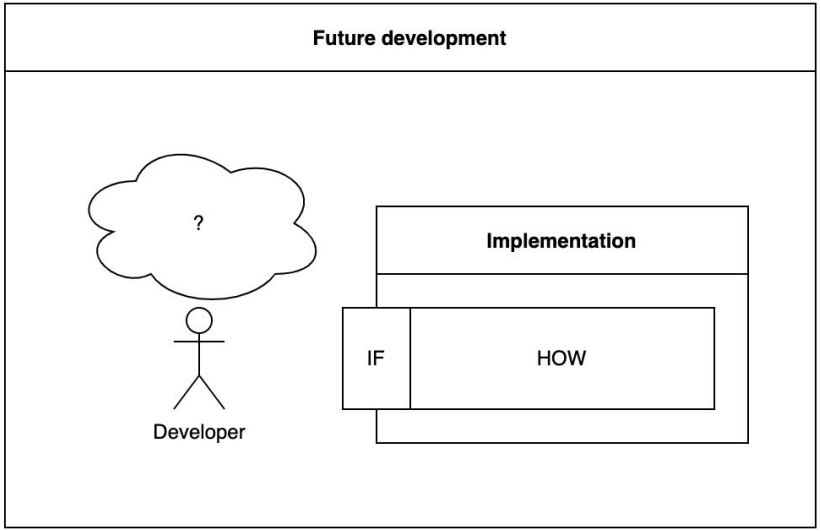
Pitfall 2: No WHAT

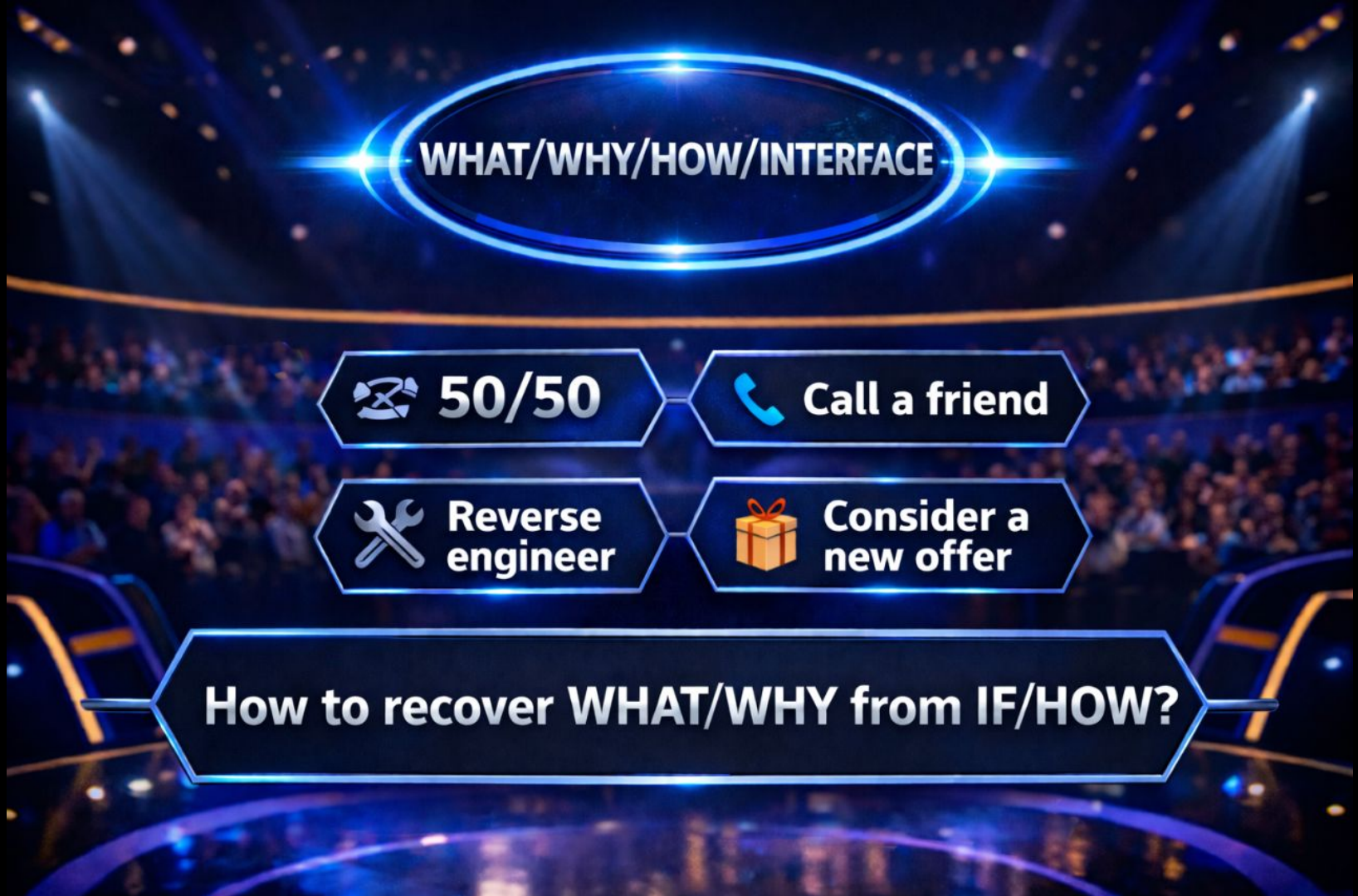
WHAT is lost and/or absorbed by HOW.

It often comes together with No WHY. Full loss of Intent.

If we are forgetting about the "WHY", why not forget about the "WHAT" as well?

WWH applied to reverse engineering (no WHAT/WHY)



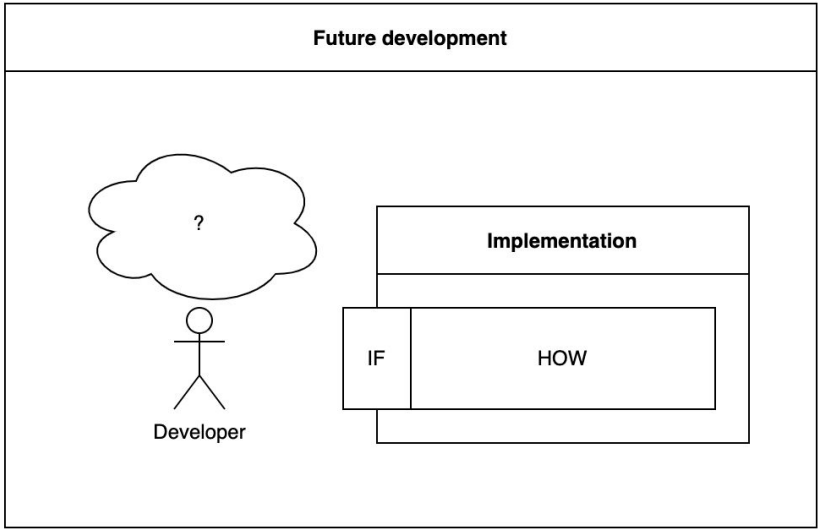

WHAT/WHY/HOW/INTERFACE
50/50
Call a friend
Reverse engineer
Consider a new offer
How to recover WHAT/WHY from IF/HOW?

No WHAT/WHY – A bold response

50/50	Call a friend
Reverse engineer	Consider a new offer

"I don't need any WHAT/WHY, the source of truth is code anyway." —

Not uncommon for a software engineer.



Pitfall 3: WHAT absorbs HOW (micromanagement)

Too much HOW in WHAT:

- Clarity and traceability to WHY is lost.
- The implementation is over-prescribed, the freedom of the HOW-implementor is limited.

Pitfall 4: HOW is reduced to INTERFACE

Seen at the documentation level:

HOW is reduced to INTERFACE only (e.g., lots of Doxygen I/F descriptions, not enough useful hints at the design).

Pitfall 5: INTERFACE is underdeveloped

How-to-use INTERFACE documentation is mixed with HOW implementation details.

Pitfall 6: WHAT-WHY-HOW recursion is not understood

Possible issues:

- The HOW→WHY recursive transitions are unclear.
- Levels of customer-supplier work are unclear or which artefacts produced by which processes.

Example: Not understanding the HLR (before design) and LLR (after/during design) requirements separation:

https://en.wikipedia.org/wiki/CAST-15

Works well with the other pitfalls.

Pitfall 7: No alignment within the team

Two teams work on the same requirements set but follow different WHAT-WHY-HOW philosophies

Example: It could be acceptable to agree that WHAT always somewhat micromanages HOW, but it must be applied consistently.

Key takeaways

Key takeaways (1)

Learn to recognize the WHAT-WHY-HOW aspects in the engineering work.

Recognize when one or more aspects is missing or mixed with other aspects

Key takeaways (2) — Importance of WHAT-WHY

The WHAT/WHY are often neglected. HOW-only is unlikely to adequately replace and represent the WHAT and WHY.

Ideally every work item should be traceable to its WHAT and WHY.

Without a formal WHAT aspect →

- Any rigorous or critical software development is weaker, if not impossible →
- See Pete Brink's ELISA webinar on requirements

Key takeaways (3) — WHAT-WHY-HOW applications

- Systems engineering, requirements, architecture
- Source code and source code comments
- Commits and pull/merge requests
- Documentation
- Technical meetings and discussions

Thank you for your attention!

Contact information:

- Stanislav Pankevich
- strictdoc.project@gmail.com

StrictDoc project:

- https://github.com/strictdoc-project/strictdoc
- Join StrictDoc Office Hours, every Tuesday 17:00-18:00 CET

Backup slides:

- Heuristics on not mixing WHAT-WHY-HOW
- More applications of WHAT-WHY-HOW
- Quizzes
- Useful resources

Backup slides

Backup slides → WHAT-WHY-HOW extras

This webinar is a shorter version of a larger WHAT-WHY-HOW training, which includes quizzes and more detailed discussions. The Backup section contains additional slides from the full training.

Heuristics on not mixing WHAT-WHY-HOW

WHY vs WHAT — Problem vs Contract

WHY — Needs, WHAT — Formal requirements

Description of problem → System obligations

Mixed WHAT/WHY:

❌ The system shall validate requirements quickly so engineers can iterate faster.

Fix:

✅ WHY: Engineers need fast feedback during editing.

✅ WHAT: The system shall validate modified requirements within 1 second.

WHY is about Problems, not Behavior

Needs should describe problems, not behavior.

❌ Malformed WHY:

The system needs incremental validation.

✅ Good WHY:

Engineers need fast feedback when editing requirements.

Shall belong to WHAT and nowhere else

- Shall statements belong to the WHAT domain only and nowhere else.
  - Example: The system shall validate modified requirements within 1 second during interactive editing.
- WHYs are Needs:
  - Example: Reviewers need to understand the intent of changes quickly.
- I/F descriptions describe the interface surface.
  - Example: The validation service exposes a POST /validate REST endpoint that accepts modified requirements and returns validation diagnostics in JSON format.
- HOW: Architecture, design, implementation details.
  - Example: The component stores requirements in a doubly linked list to support constant-time insertion and removal during incremental validation.

WHAT vs HOW

- Rule of thumb: WHAT — Requirements, HOW — Design
- Boundary and transition between: System obligations → Concrete realization
- With WHAT:
  - Multiple solutions possible
  - Can be implemented by multiple teams
- With HOW:
  - Irreversible specific solution appears
- WHAT vs HOW test:
  - Could multiple designs satisfy this statement? If no, it probably contains HOW.

HOW: Interface vs Implementation

Boundary and transition between:

- External surface of solution → Internal implementation of solution

Implementation:

- If possible to change implementation without breaking outside surface

Interface:

- If the change breaks external contract

WHAT/WHY in HOW

- Relative to Requirements/WHAT, Architecture and Design is HOW
- But:
  - Design Definition File → The WHAT of HOW
  - Design Justification File → The WHY of HOW
- Architecture Design Records (ADRs):
  - Decision is WHAT
  - Justification is WHY
  - Performed trade-offs is HOW

More applications of WHAT-WHY-HOW

Applications — Git commits

```
fix validator
```

VS

```
feat(validator): add incremental validation for changed requirements

WHAT: validate only changed requirements
WHY: full validation is slow on large projects
HOW: detect changed nodes and revalidate dependencies
```

Applications — Pull/merge requests

MR: fixes + improvements

Refactored the validator and cleaned up some stuff.

Also improved performance a bit and fixed a couple of issues related to validation that sometimes happened when editing requirements.

Other small changes:

- refactoring
- improved error messages
- added tests
- some performance improvements
- minor fixes

Applications — Pull/merge requests

MR: Incremental validator

WHAT

Validator now runs incrementally instead of validating the entire requirement graph.

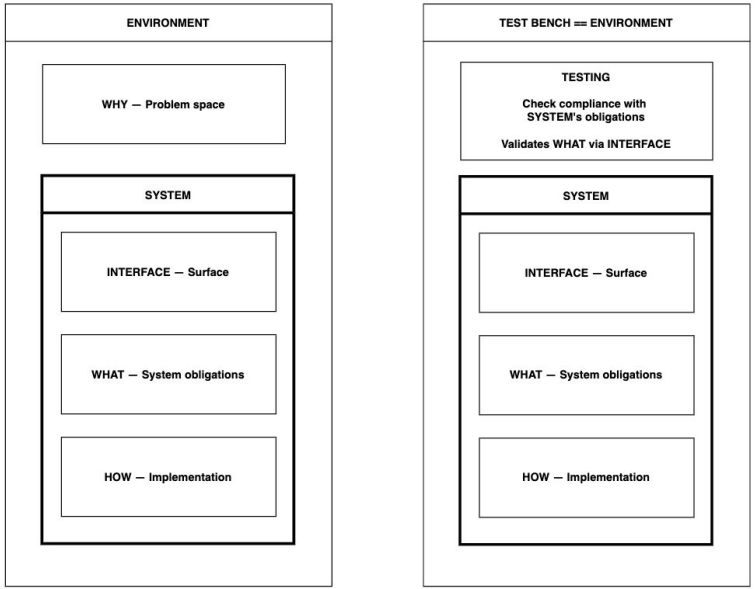
WHY

Full validation is slow (~20s on bigger projects) and makes editing painful.
Most edits touch only a few requirements.

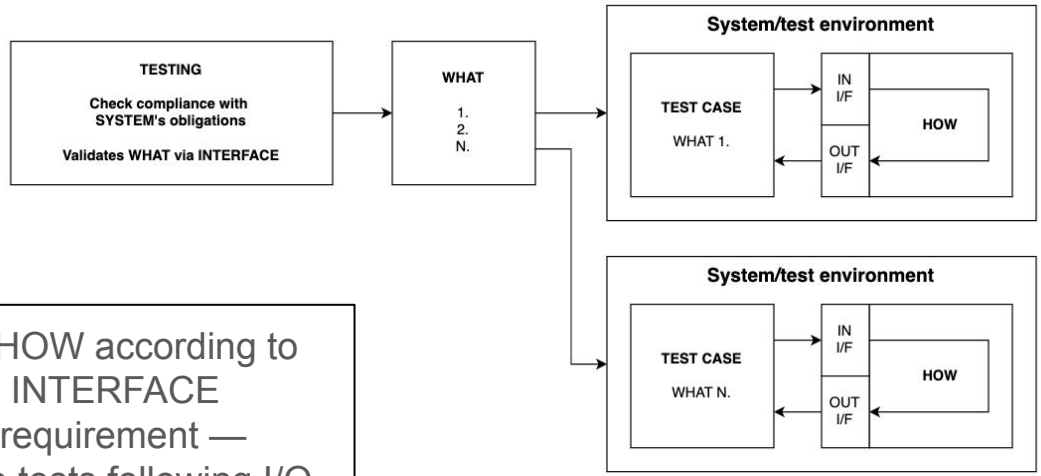
HOW

- Track changed requirements
- Revalidate affected nodes
- Split validator into smaller passes
- Improve diagnostics
- Add tests

WWH(I) applied to testing: Environment → Test bench



WWH(I) applied to testing: System under test



- Test validates HOW according to WHAT through INTERFACE
- Good testable requirement — Natural to write tests following I/O model

Quizzes

Quiz: Which aspect is covered by Linux man pages?

A. WHAT	B. WHY
C. INTERFACE	D. HOW

```
man sem_wait
```

NAME

sem_wait, sem_timedwait, sem_trywait - lock a semaphore

LIBRARY

POSIX threads library (libpthread, -lpthread)

SYNOPSIS

```
#include <semaphore.h>

int sem_wait(sem_t *sem);
int sem_trywait(sem_t *sem);
int sem_timedwait(sem_t *restrict sem,
                  const struct timespec *restrict abs_timeout);
```

Feature Test Macro Requirements for glibc (see **feature_test_macros**(7)):

sem_timedwait():
_POSIX_C_SOURCE >= 200112L

DESCRIPTION

sem_wait() decrements (locks) the semaphore pointed to by sem. If the semaphore's value is greater than zero, then the decrement proceeds, and the function returns, immediately. If the semaphore currently has the value zero, then the call blocks until either it becomes possible to perform the decrement (i.e., the semaphore value rises above zero), or a signal handler interrupts the call.

sem_trywait() is the same as **sem_wait**(), except that if the decrement cannot be immediately performed, then call returns an error (errno set to **EAGAIN**) instead of blocking.

sem_timedwait() is the same as **sem_wait**(), except that abs_timeout specifies a limit on the amount of time that the call should block if the decrement cannot be immediately performed. The abs_timeout argument points to a **timespec**(3) structure that specifies an absolute timeout in seconds and nanoseconds since the Epoch, 1970-01-01 00:00:00 +0000 (UTC).

If the timeout has already expired by the time of the call, and the semaphore could not be locked immediately, then **sem_timedwait**() fails with a timeout error (errno set to **ETIMEDOUT**).

If the operation can be performed immediately, then **sem_timedwait**() never fails with a timeout error, regardless of the value of abs_timeout. Furthermore, the validity of abs_timeout is not checked in this case.

RETURN VALUE

All of these functions return 0 on success; on error, the value of the semaphore is left unchanged, -1 is returned, and errno is

Quiz: Which aspect is covered by Doxygen?

A. WHAT	B. WHY
C. INTERFACE	D. HOW

Doxygen conventions are designed to cover the INTERFACE. They sometimes cover HOW, while rarely addressing WHAT and WHY, although Doxygen supports requirements tracing these days.

Doxygen example — Interface

```c
/**
 * @brief Calculates the sum of two integers.
 *
 * Adds two integer values and returns the result.
 *
 * @param a First integer value.
 * @param b Second integer value.
 *
 * @return The sum of a and b.
 */
int add(int a, int b);
```

Quiz: Which aspect is covered by Interface Requirements Document (IRD)?

A. WHAT	B. WHY
C. INTERFACE	D. HOW

Example: Interface requirement and description

WHAT (Functional requirement):

- When the user activates the Play control, the system shall start media playback from the current position.

WHAT (Interface requirement):

- The control panel shall provide a Play control that allows the operator to start playback.

HOW (Interface description):

- The control panel provides a round Play button located in the playback control section of the user interface.
- The Play button is colored green and labeled with the standard ▶ symbol to indicate playback start.

Useful resources

Useful resources

Space:

- ECSS Software engineering handbook (ECSS-E-HB-40A)
- NASA Systems Engineering Handbook and the Expanded Guidance handbooks

Aviation:

- Leanna Rierson Developing Safety-Critical Software: A Practical Guide for Aviation Software and DO-178C Compliance
- Requirements Engineering Management Handbook (DOT/FAA/AR-08/32)
- Merging High-Level and Low-Level Requirements (CAST-15 position paper)

Systems engineering:

- Systems Engineering Fundamentals by DOD Systems Management College
- Systems and software engineering — System life cycle Processes (ISO/IEC 15288)
- Software engineering body of knowledge by IEEE CS (SWEBOK)